

Algoritmusok és adatszerkezetek II.

12. gyakorlat

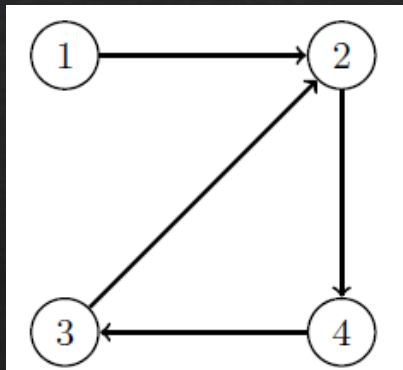
Tartalom:

Tranzitív lezárt
Mintaillesztés

- ◊ Tranzitív lezárt
- ◊ Mintaillesztés feladat
- ◊ Brute Force algoritmus
- ◊ KMP algoritmus
- ◊ Quick Search algoritmus

Tranzitív lezárt

- ◊ Feladat:
- ◊ Adott egy $G=(V,E)$ $V=\{1,2,...,n\}$ irányított gráf, minden egyes $i,j \in V$ csúcspárra tudni szeretnénk, hogy létezik-e út i -ből j -be.
- ◊ **G tranzitív lezártja definíció:**
Egy $G(V, E)$ gráf tranzitív lezártja egy $G'(V, E')$ gráf, ahol $(u, v) \in G'.E'$ akkor és csak akkor, ha G -ben vezet út u -ból v -be.



G	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	0	1	0	0
4	0	0	1	0

csúsmátrixos ábrázolás

G'	1	2	3	4
1	1	1	1	1
2	0	1	1	1
3	0	1	1	1
4	0	1	1	1

tranzitív lezárt csúsmátrixos ábrázolása

Irányítatlan gráfra is értelmezhető.

Tranzitív lezárt

♦ Megoldási lehetőségek:

- ❑ Mátrix szorzáson alapuló megoldás: ha $A[n][n]$ mátrix ábrázolja a gráfot, A^2 ábrázolja a pontosan 2 élből álló utakat ($A^2[i,j] = d$, ha d a 2 hosszú utak száma i -ből j -be),
 A^3 hasonlóan a 3 élből, A^{n-1} az $n-1$ élből álló utakat ábrázolja $\rightarrow$ ezekből előállítható a tranzitív lezárt. (Költséges lenne: $\Theta(n^4)$)
- ❑ Bejárás (szélességi, mélységi) indítása minden csúcsból, mint kezdőcsúcsból.
 $O(n \cdot (n+m))$
- ❑ Floyd-Warshall algoritmus lefuttatása, élek súlyának egyet választva.
Ha $D[i,j] < n$ akkor van út i -ből j -be, ha $D[i,j] = \infty$, akkor nincs. $\Theta(n^3)$
- ❑ Floyd-Warshall átalakításával kapott Warshall algoritmus.

Tranzitív lezárt, Warshall algoritmus

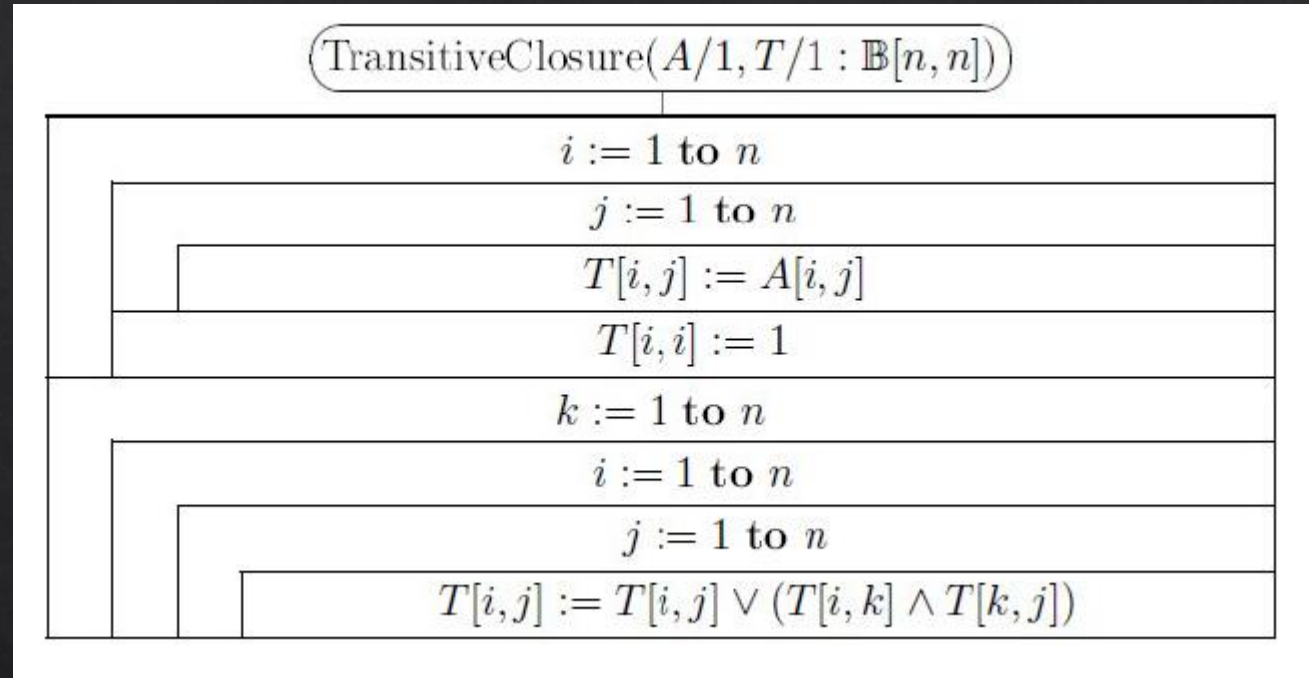
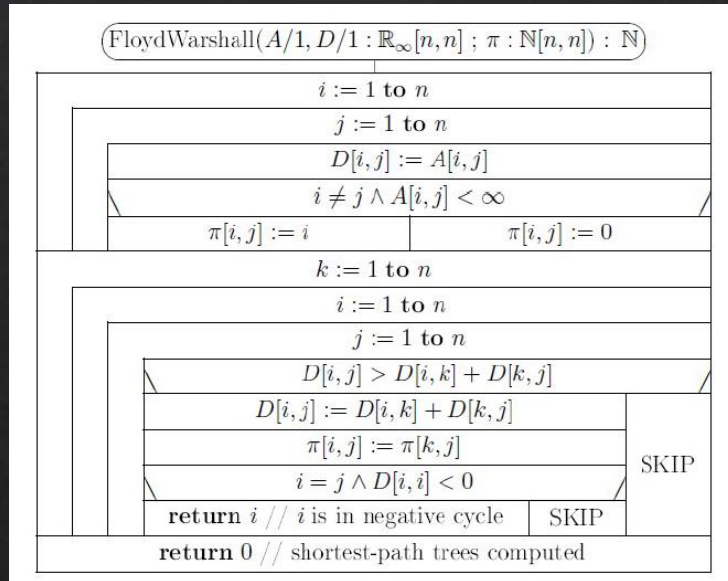
- ♦ A Floyd-Warshall algoritmus átalakítása:
- ♦ $T^{n \times n}$ – es logikai mátrixot használunk (1-igaz, 0-hamis).
- ♦ A rekurzív képlet definíciója :

$$T_{ij}^{(0)} = \begin{cases} 1, & \text{ha } i = j \vee (i, j) \in G.E \\ 0, & \text{különben} \end{cases}$$

$$T_{ij}^{(k)} = T_{ij}^{(k-1)} \vee (T_{ik}^{(k-1)} \wedge T_{kj}^{(k-1)})$$

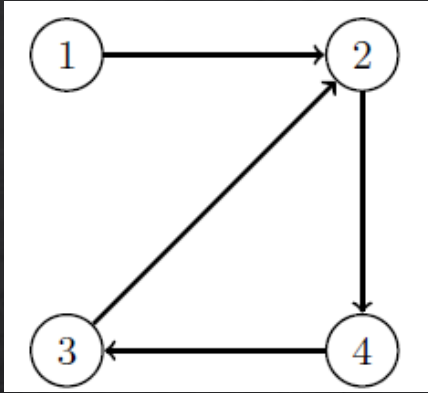
- ♦ A közelítés átalakítása: a $+$ művelet helyett $\wedge$, a min helyett $\vee$

Tranzitív lezárt, Warshall algoritmus



Műveletigény: $\Theta(n^3)$

Tranzitív lezárt, Warshall algoritmus



$T^{(0)}$	1	2	3	4
1	1	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	0	1	1

$T^{(1)}$	1	2	3	4
1	1	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	0	1	1

$T^{(2)}$	1	2	3	4
1	1	1	0	1
2	0	1	0	1
3	0	1	1	1
4	0	0	1	1

$T^{(3)}$	1	2	3	4
1	1	1	0	1
2	0	1	0	1
3	0	1	1	1
4	0	1	1	1

$T^{(4)}$	1	2	3	4
1	1	1	1	1
2	0	1	1	1
3	0	1	1	1
4	0	1	1	1

Mintaillesztés

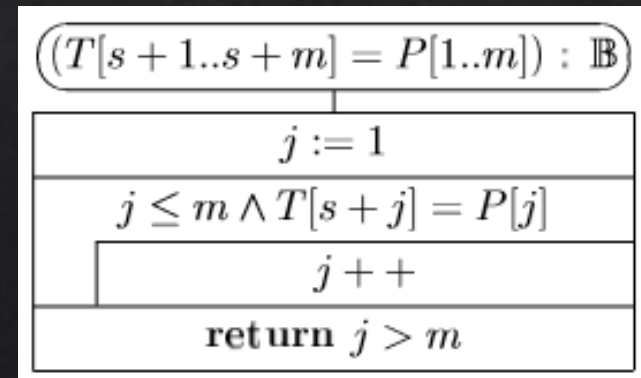
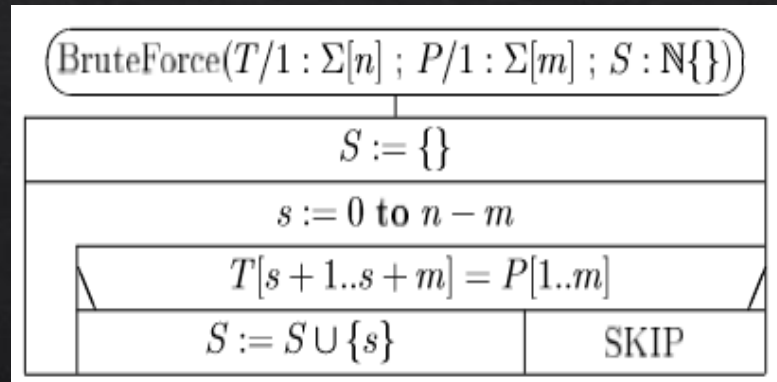
- ◆ Feladat:
Egy szövegben egy minta összes előfordulását keressük.
- ◆ Jelölések:
 - szöveg: $T[1..n]$
 - minta: $P[1..m]$
 - Érvényes eltolások halmaza (eredmény): S
 $s \in S \Leftrightarrow$ a minta illeszkedik $T[s+1]..T[s+m]$ pozíción.
 - Ábécé betűi: $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_d\}$

Mintaillesztés

- ◆ Mintaillesztő algoritmusok összehasonlításánál két fontos kérdéssel foglalkozunk:
- ◆ Műveletigény (hány összehasonlítást végzünk el)
- ◆ Mennyire „ugrál” a szövegben?
Ez azért fontos, mert előfordulhat, hogy a mintaillesztést olyan adatszerkezetre kell megoldani (pl. szekvenciális fájl, bináris fa), ahol nem megengedett művelet az indexelés. Ekkor az olyan algoritmusoknál, ahol a szövegben vissza, vagy előre „ugrunk”, puffer segítségét kell igénybe venni.
- ◆ Vizsgált algoritmusok:
 - ▢ Brute-Force
 - ▢ KMP
 - ▢ QuickSearch

Brute Force

- ♦ A Brute-Force (nyers erő) algoritmus nem más, mint egy kiválogatásba ágyazott optimista eldöntés.
 - ☐ Az algoritmust szemléletesen úgy lehet elképzelni, mintha a mintát tartalmazó sablont tolnánk végig a szövegen, és balról jobbra ellenőrizzük, hogy a minta karakterei egyeznek-e a lefedett szöveg karaktereivel.
 - ☐ Amennyiben nem egyező karakterpárt találunk, a mintát eggyel jobbra toljuk a szövegen, és megint kezdjük a minta elejéről az összehasonlítást.



Brute Force

◆ Műveletigény:

- ❑ Legjobb eset: A minta első karaktere nincs a szövegben.
Ekkor $m\ddot{O}(n,m) = n - m + 1 \in \Theta(n)$
- ❑ Legrosszabb eset: A minta minden eltolásnál csak a minta utolsó karakterénél romlik el az illeszkedés. Ekkor $M\ddot{O}(n,m) = (n - m + 1) * m \in \Theta(n*m)$

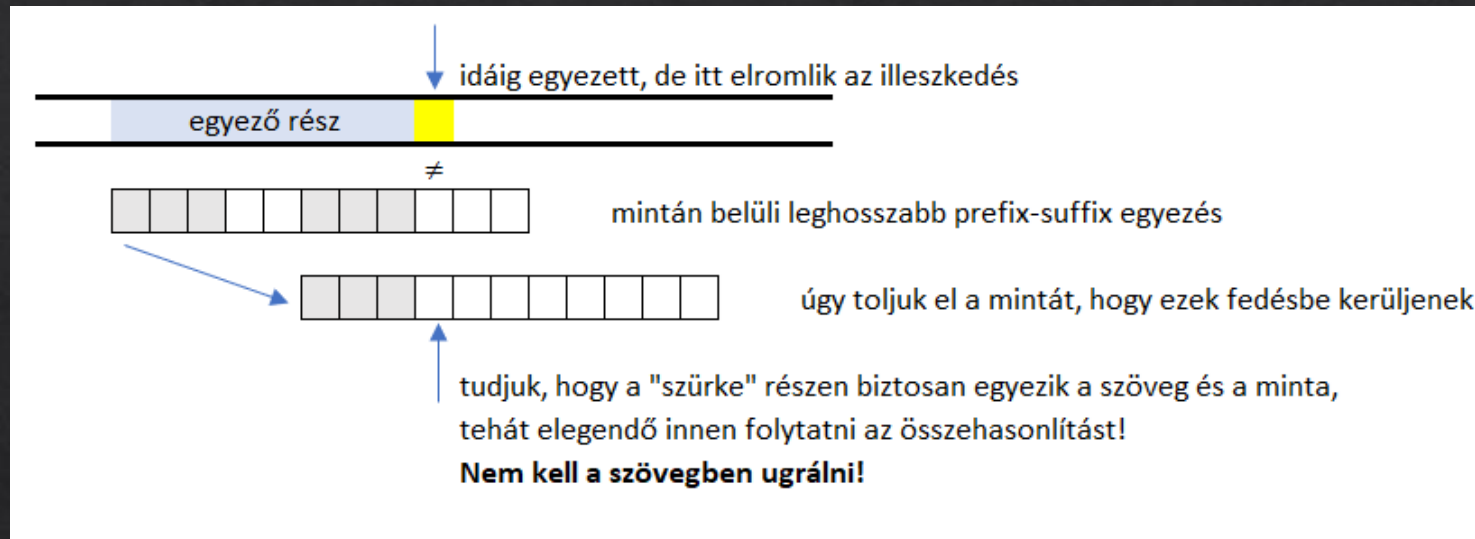
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
T=		A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	
		A	A	A	B													
			A	A	A	B												
				A	A	A	B											

◆ Szekvenciális sorozatokra, fájlokra való alkalmazhatóság:

- ❑ A szövegben „ugrálunk”, ezért az olyan adatszerkezeteknél ahol nem megengedett az indexelés, szükség van puffer használatára.

KMP

- ◆ Knuth-Morris-Pratt algoritmus
- ◆ Az ötlet:



- ◆ A KMP algoritmusnál nem szükséges minden esetben a minta elejéről kezdeni az illeszkedés vizsgálatát (a vizsgált betű indexe vagy ugyanaz marad, vagy egyet jobbra lép).
- ◆ Amennyiben a mintát úgy „toljuk el” a szöveg alatt, hogy a minta kezdőszelete (prefixe) egy valódi végszeletnél (suffix) kezdődjön, azaz a prefix a szuffixel kerüljön fedésbe, a prefixet már nem kell újra vizsgálni.
- ◆ Ehhez a minta előfeldolgozására van szükség.

KMP, a minta előfeldolgozása

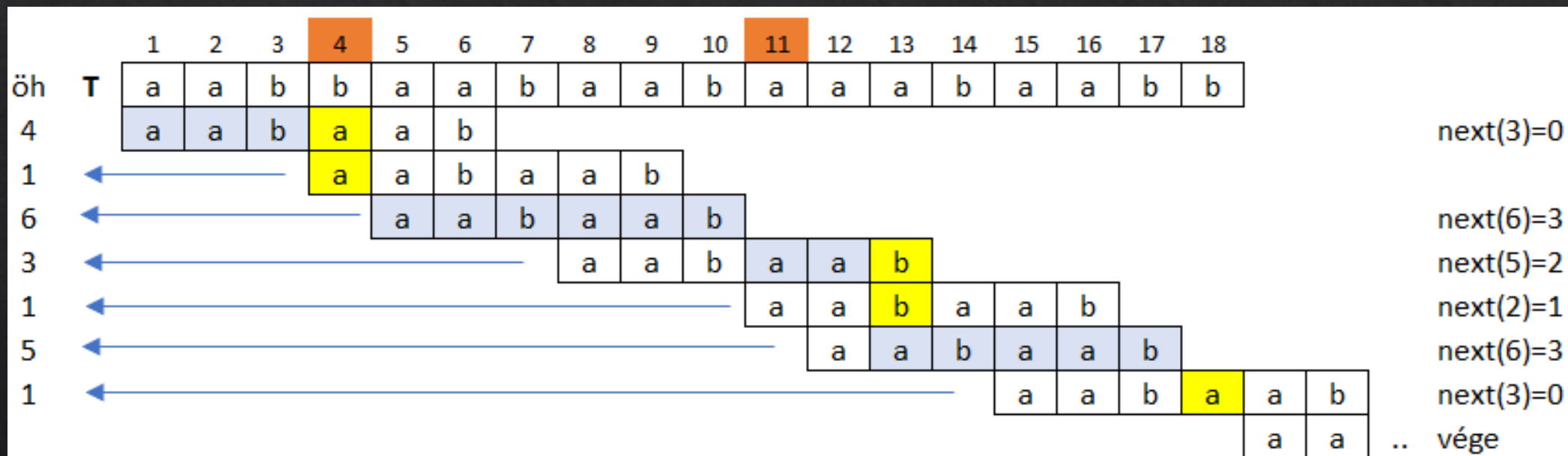
- ◊ Az előfeldolgozás során definiálunk egy next függvényt, amely megadja a minta egyes kezdőrészleteire a leghosszabb egymással egyező prefix-suffix párok hosszát.
- ◊ Ezt felhasználva tudjuk megadni a minta eltolásának mértékét, ha az illeszkedés elromlik.
- ◊ Fontos tulajdonságok:
 - ◻ $next(j) \in 0..j-1, j \in 1..m$
 - ◻ $next(j+1) \leq next(j) + 1$ ($j \in 1..m-1$), azaz next függvény maximum egyesével növekszik

Minta:	1	2	3	4	5	6	7	8
	a	b	a	b	a	c	a	b
Next:	1	2	3	4	5	6	7	8
	0	0	1	2	3	0	1	2
a	1	2	3	4	5	6	7	8
	0							
a b	1	2	3	4	5	6	7	8
	0	0						
a b a	1	2	3	4	5	6	7	8
	0	0	1					
a b a b	1	2	3	4	5	6	7	8
	0	0	1	2				
a b a b a	1	2	3	4	5	6	7	8
	0	0	1	2	3			
a b a b a c	1	2	3	4	5	6	7	8
	0	0	1	2	3	0		
a b a b a c a	1	2	3	4	5	6	7	8
	0	0	1	2	3	0	1	
a b a b a c a b	1	2	3	4	5	6	7	8
	0	0	1	2	3	0	1	2

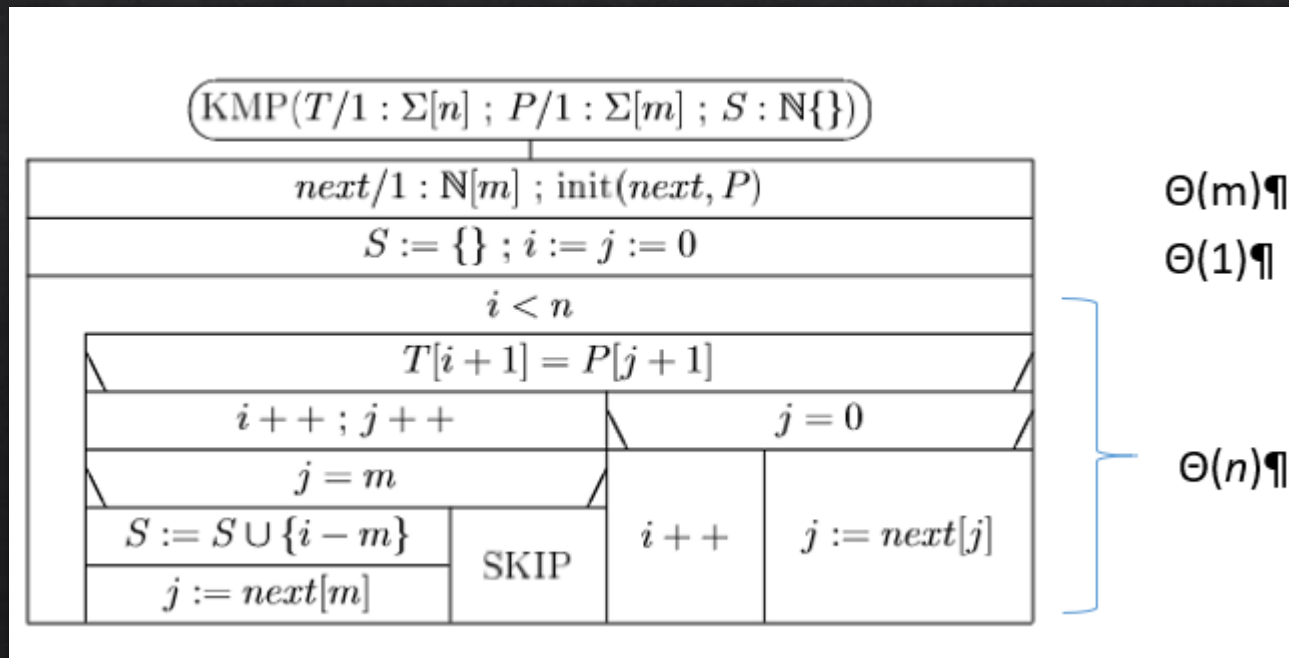
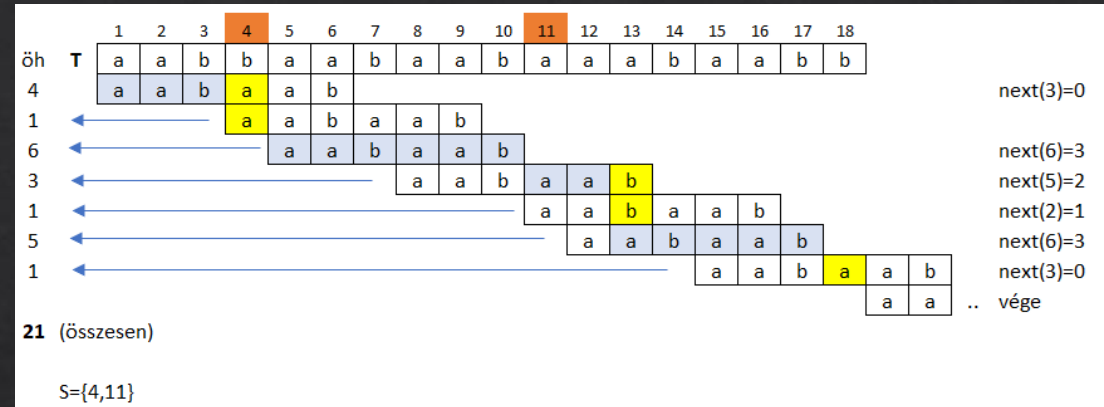
KMP, az algoritmus működése

	1	2	3	4	5	6
P	a	a	b	a	a	b

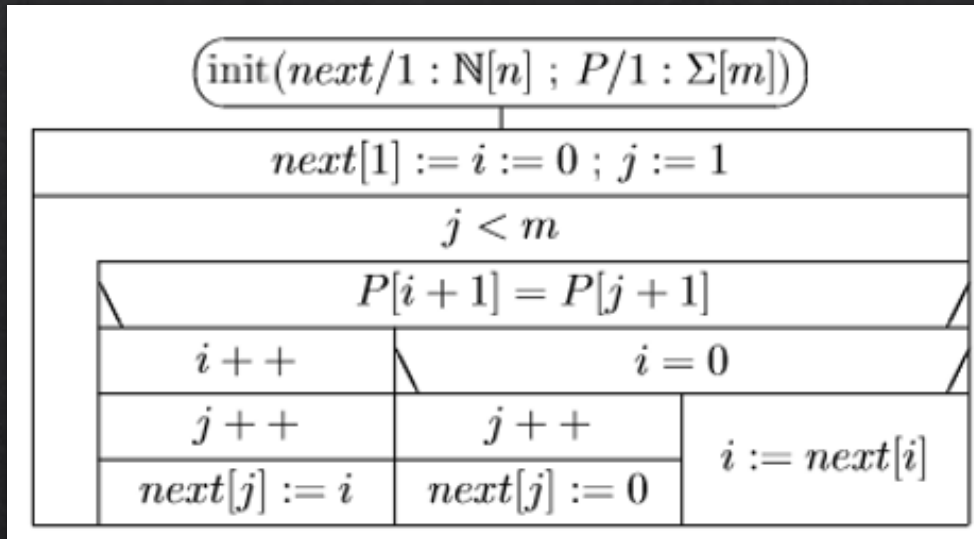
	1	2	3	4	5	6
Next	0	1	0	1	2	3



KMP algoritmus struktogramja és műveletigénye



KMP, $\text{init}(\text{next}, P)$ működése



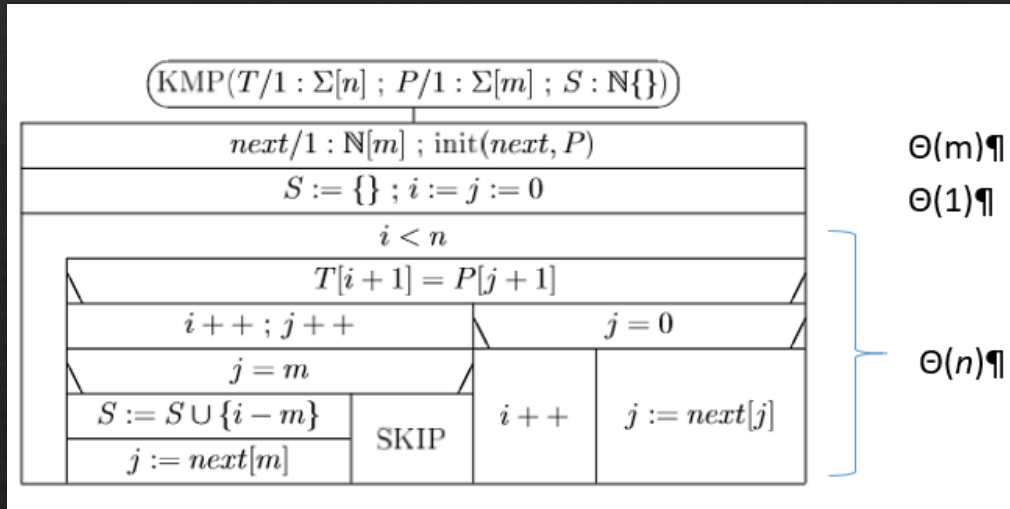
init műveletigény: $\Theta(m)$

$p[1] \neq p[2]$
 $p[1] = p[3]$
 $p[2] = p[4]$
 $p[3] = p[5]$
 $p[4] \neq p[6]$
 $p[2] \neq p[6]$
 $p[1] \neq p[6]$

1	2	3	4	5	6		i	j	next
a	b	a	b	a	c				
							0	1	next[1]=0
	a	b	a	b	a	c	0	2	next[2]=0
		a	b	a	b	ac	1	3	next[3]=1
		a	b	a	b	ac	2	4	next[4]=2
		a	b	a	b	ac	3	5	next[5]=3
		a	b	a	b	ac	1	5	$i = \text{next}[3]$
				a	b	abac	0	5	$i = \text{next}[1]$
					a	babac	0	6	next[6]=0

	1	2	3	4	5	6
next:	0	0	1	2	3	0

KMP hatékonyság



- ◆ Az init műveletigénye $\Theta(m)$. (Ahol m a minta hossza.)
- ◆ Tegyük fel, hogy $m \ll n$, ekkor a KMP műveletigénye legjobb és legrosszabb esetben is $\Theta(n)$.
- ◆ **LINEÁRIS idő!**
- ◆ $T \in \Omega(n)$, mivel i növekedni egyesével tud, és n -ig nő $\Rightarrow$ biztos van n lefutás
- ◆ $T \in O(n)$ mivel a fő ciklus max $2n$ -szer fut le.

Előny: A szövegben nem kell „visszaugrani”. Ennek jelentősége például a szekvenciális sorozat/fájl formában adott szövegnél van, mivel ekkor puffer használata nélkül is tudjuk alkalmazni a KMP algoritmust.

Önálló feladat

◆ KMP algoritmus lejátszása:

📄 KMP_pelda.xlsx

$P[j]=$	B	A	B	A	B	A	B
$j=$	1	2	3	4	5	6	7
$next[j]=$	0	0	1	2	3	4	5

A $P[1..7] = BABABAB$ mintát keressük a $T[1..22] = BABBABABAB ABBABABABAAB$ szövegben.

$i=$	H.SZ.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
$T[i]=$		B	A	B	B	A	B	A	B	A	B	A	B	B	A	B	A	B	A	B	A	A	B
	4	<u>B</u>	<u>A</u>	<u>B</u>	<u>A</u>																		
	1			B	A																		
$s=3$	7				<u>B</u>	<u>A</u>	<u>B</u>	<u>A</u>	<u>B</u>	<u>A</u>	<u>B</u>												
$s=5$	2						B	A	B	A	B	<u>A</u>	<u>B</u>										
	1								B	A	B	A	B	A									
	1										B	A	B	A									
	1											B	A										
$s=12$	7												<u>B</u>	<u>A</u>	<u>B</u>	<u>A</u>	<u>B</u>	<u>A</u>	<u>B</u>				
	2															B	A	B	A	B	<u>A</u>	<u>B</u>	
	1																B	A	B	A	<u>B</u>		
	1																		B	A	<u>B</u>		
	1																				<u>B</u>		
	1																						<u>B</u>

$S=\{3; 5; 12\}$

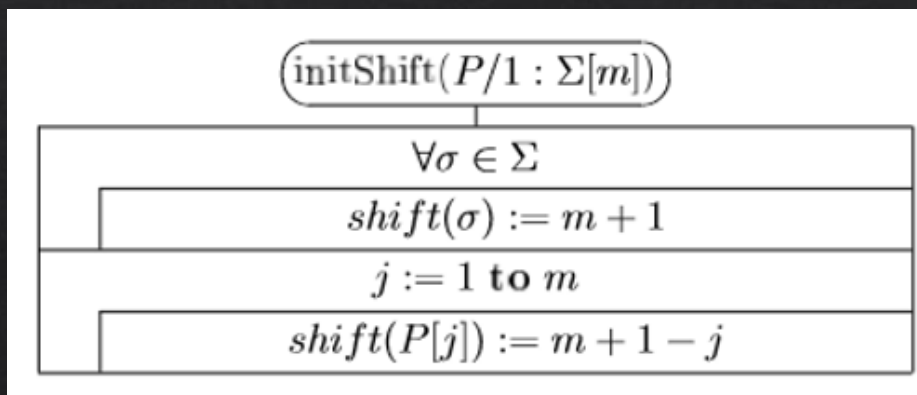
Quick Search algoritmus

- ♦ Ötlet:
- ♦ Ha elromlik az illeszkedés, akkor nézzük a szövegben a minta utáni első karaktert, és úgy toljuk el a mintát, hogy illeszkedjen a szöveg ezen karakteréhez.
- ♦ Ha a mintában nem szerepel ez a karakter, akkor átugorhatjuk a mintával ezt a pozíciót.
- ♦ Az új vizsgálatot mindig a minta elejétől kezdjük.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	a	a	b	a	c	e	b	a	d	a	c	b	a	b
a	b	a	c											
		a	b	a	c									
				a	b	a	c							
						a	b	a	c					
							a	b	a	c				
								a	b	a	c			
									a	b	a	c		
										a	b	a	c	
											a	b	a	c

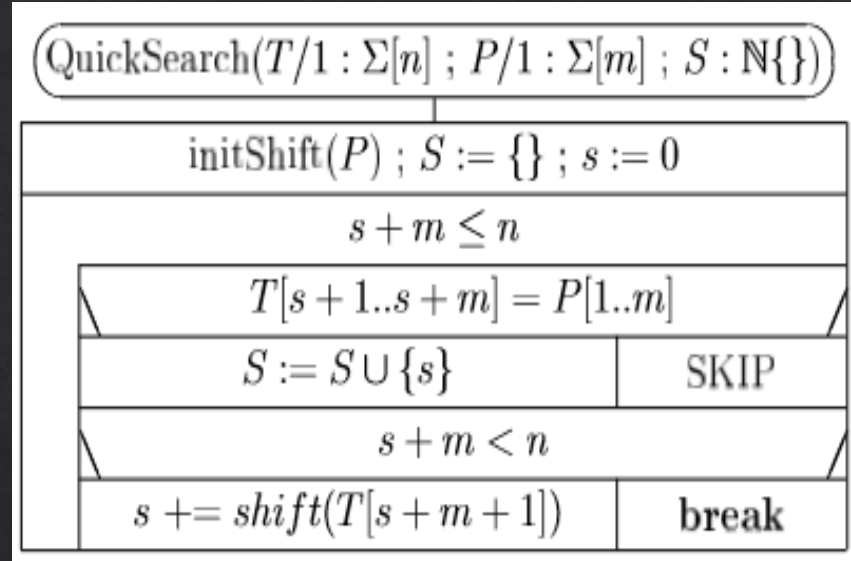
shift függvény

- ♦ A minta eltolásának végrehajtásához bevezetjük a shift függvényt.
- ♦ A shift függvény az ABC minden betűjére megadja az eltolás nagyságát, amelyet akkor alkalmazunk, ha vagy elromlik az illeszkedés, vagy sikerült a minta illesztése egy adott indexnél.
- ♦ Ekkor megnézzük mennyi a shift() értéke annak a betűnek, ami a minta után következik a szövegben, és a vizsgálat kezdetét meghatározó változó megnöveljük vele.

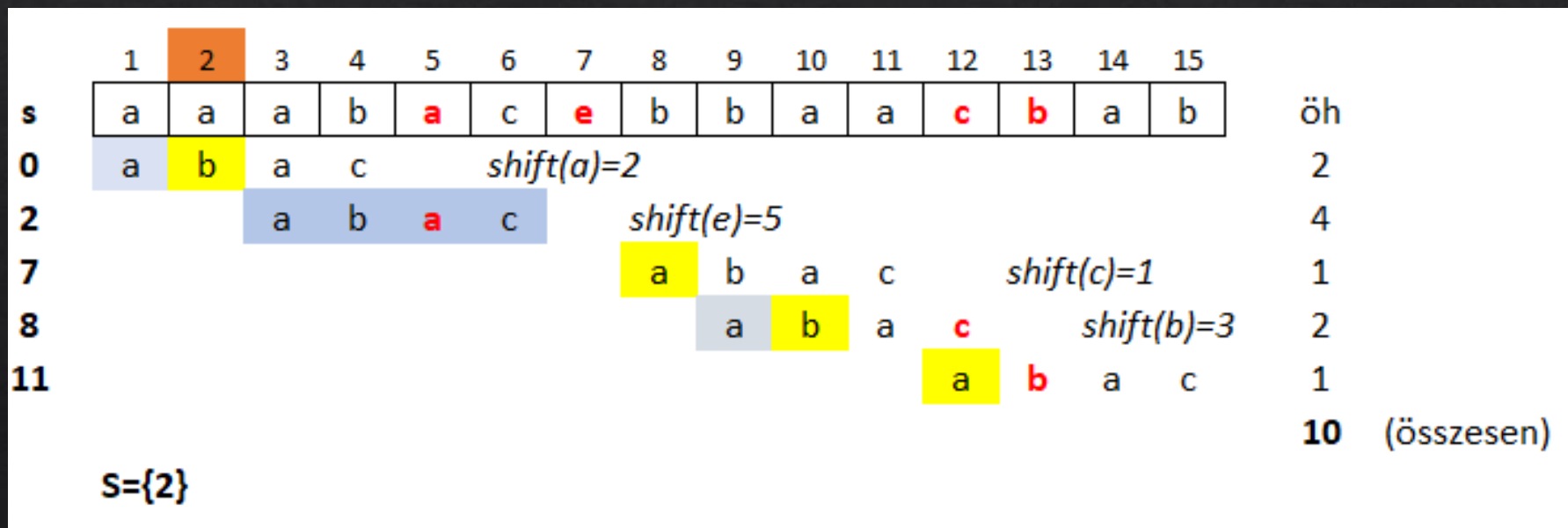


P =	a	b	a	c	
Σ =	a	b	c	d	e
shift	a	b	c	d	e
	5	5	5	5	5
a	4				
b		3			
a	2				
c			1		
végül:	2	3	1	5	5

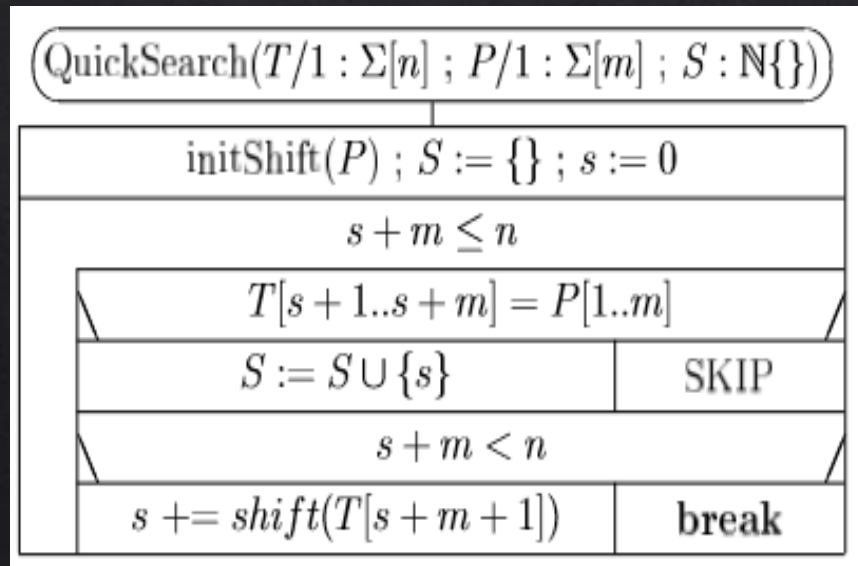
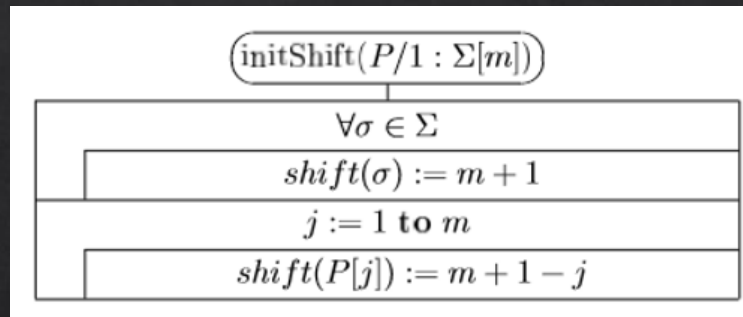
QuickSearch algoritmus és működése



break = s:=s+1



Quick Search műveletigénye



- ♦ initShift műveletigény:
 $\Theta(d) + \Theta(m) = \Theta(m)$
 d - az ábécé elemszáma, ami konstans
- ♦ QuickSearch műveletigény:
- ♦ legjobb eset:
 $MT(n, m) \in \Theta(n/(m+1)) = \Theta(n/m)$
 (A minta első karakterénél már elromlik az illeszkedés, továbbá a minta utáni karakter sem fordul elő a mintában, így azt „átugorjuk”.)
- ♦ legrosszabb eset:
 $MT(n) = \Theta((n-m+1)*m) \in \Theta(n*m)$
 (A minta végén romlik el az illeszkedés, és kicsi az „ugrás”. Például: csupa ,a’-ban keressük az aaab mintát.)

Önálló gyakorló feladatok

- ◊ Quick Search algoritmus lejátszása, példák:
- ◊ QS_pelda.xlsx